



University of
Applied Sciences

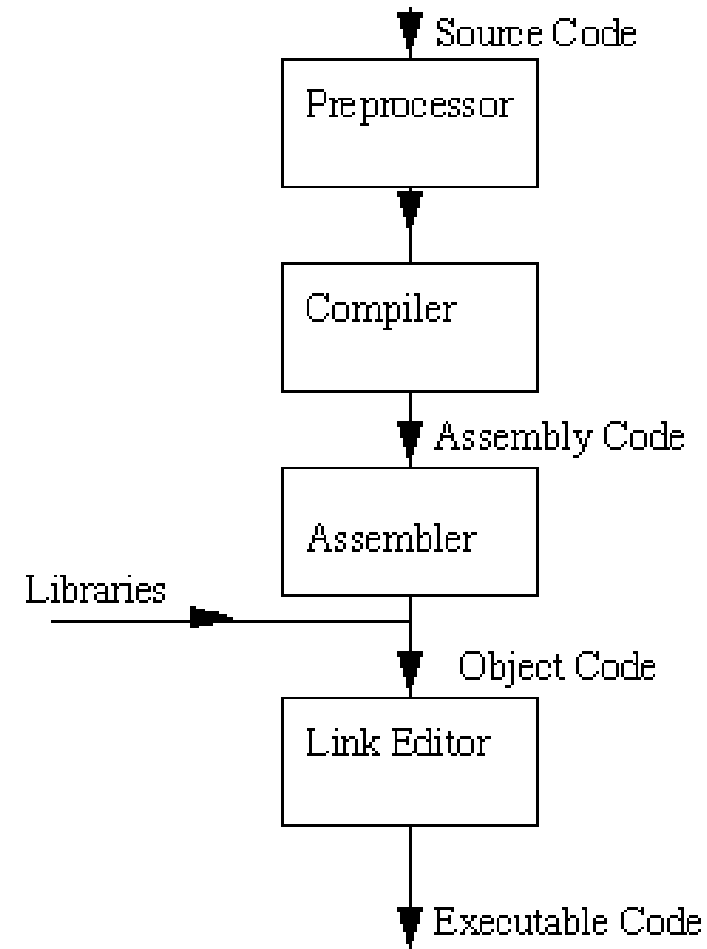
TECHNIKUM

WIEN

Intro:
C/C++ - Programming under Linux

Toolchain

- **Source Code Editor**
 - Conventions: Extension .c or .cpp
 - E.g. vi myprog.c
- **Compiler**
 - Various compilers or integrated IDEs available
 - Linux Standard: GNU C/C++ compiler gcc, g++
- **Debugging**
 - Debugging Tools under Linux: gdb, ddd, ...
- **Executing the program**
 - Conventions: no extension (.EXE)
 - ./myprog



gcc/g++

- **gcc/g++ are used for all steps from preprocessor to linker**
 - Complex handling with countless options, flags and parameters
 - For details: `man gcc`, `man g++`
- **gcc myprog.c**
 - Creates an executable named `a.out` when no errors occur
- **gcc -o myprog myprog.c**
 - Creates an executable named `myprog`
- **gcc -g -Wall -O -o myprog myprog.c**
 - Enables all warnings, code optimization and symbols for debugging
- **g++ -Wall -g -std=c++17 -o test test.cpp**
 - Compile C++ with `c++17` standard
- **g++ Cheatsheet**
 - <https://bytes.usc.edu/cs104/wiki/gcc/>

gcc

- Important options

-c	Compile only without linking
-E	Preprocessor only
-S	Generate only assembly code
-o filename	name of executable
-g	Add symbols for debugging (debug build)
-O	Enable optimizations
-Dsymbol=value	#define symbol=value
-Wall	Show all warnings
-lbibliothek e.g. -lm	Link with library Link with math library
-I directory	Include directory in search path for header files
-L directory	Include directory in search path for libraries

make

- **Powerful tool to automate building software**
- **Reads instructions from a simple textfile (Makefile) how to compile and build a program and which dependencies exist**
- **Executes all steps for a build target automatically instead of manual steps from command line with lots of compiler/linker options**
- **Make checks dependencies and timestamps, which source files changed and compiles only not up-to-date files**
- **See also**
- **https://openbook.rheinwerk-verlag.de/linux_unix_programmierung/Kap17-000.htm**

make

- **Syntax of Makefile**

```
# Comment
```

```
Target1: Dependencies
```

```
<tab>command 1
```

```
<tab>command 2
```

```
<tab>...
```

```
Target2: Dependencies
```

```
<tab>command 1
```

```
<tab>command 2
```

```
<tab>...
```

- **Invoke with:**

- **make ... builds first target in Makefile**
- **make target ... builds target**

make

- **Simple example**

```
#Makefile für myprog.c
all: myprog

myprog: myprog.c
    gcc -g -Wall -O -o myprog myprog.c

clean:
    rm -f myprog
```

- **make or make all builds program myprog**

- Target all depends on target myprog the executable
- Target myprog depends on the source file myprog.c, so when the source is updated make will build the target and invoke the gcc command

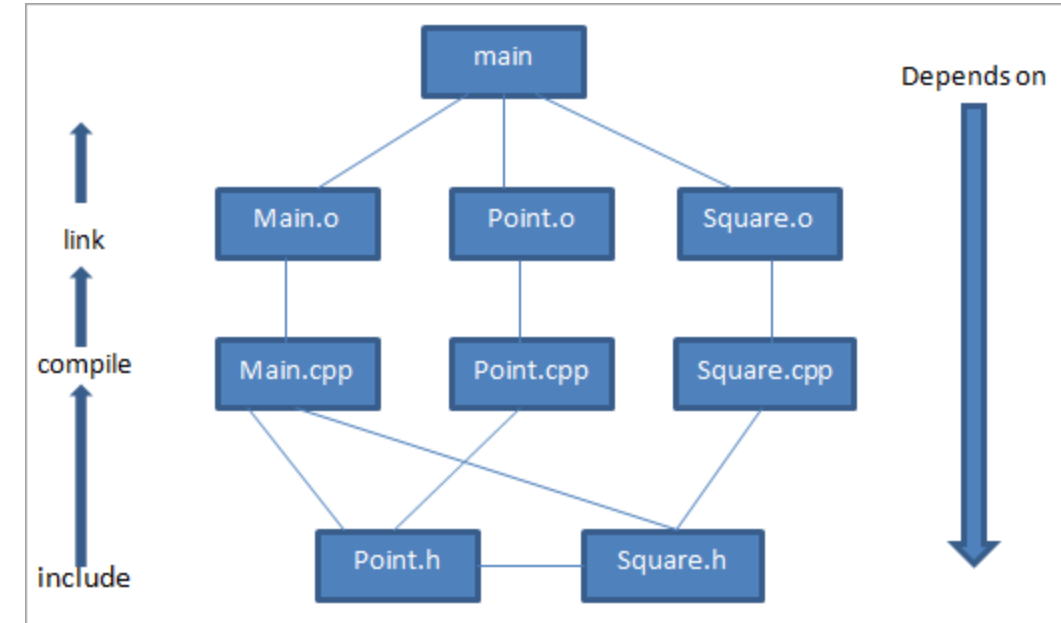
- **make clean deletes all generated files**

- **Convention: Targets all and clean should always be present and all should be first target!**

make

- **Example with variables**

```
# Makefile for project main
# *****
# Variables to control Makefile operation
CC = g++
CFLAGS = -Wall -g
# *****
all: main
main: Main.o Point.o Square.o
    $(CC) $(CFLAGS) -o main Main.o Point.o Square.o
Main.o: main.cpp Point.h Square.h
    $(CC) $(CFLAGS) -c Main.cpp
Point.o: Point.h
    $(CC) $(CFLAGS) -c Point.cpp
Square.o: Square.h Point.h
    $(CC) $(CFLAGS) -c Square.cpp
clean:
    rm -f main *.o
```



- **More examples**

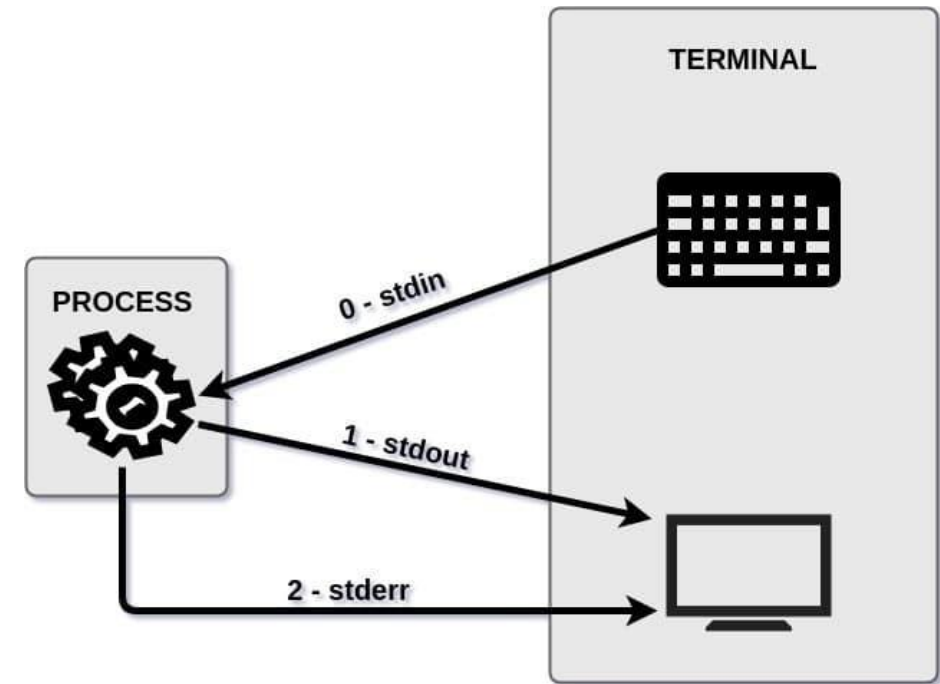
- <https://makefiletutorial.com/>

Standard Streams

- Stdin (0), stdout (1) and stderr (2) are defined as filestreams in `stdio.h`
 - C++: `cin`, `cout`, `cerr`

- More info:

<https://www.putorius.net/linux-io-file-descriptors-and-redirect.html>



General Conventions

- **Error Handling**

- Check return codes of functions
- Write meaningful error-messages including the program name (stderr not stdout)
 - C: `fprintf(stderr,...)` instead of `printf`
 - C++: `cerr` instead of `cout`
 - `fprintf(stderr,"%s: Error opening file %s\n",argv[0],filename);`

- **Compile with warnings enabled (-Wall)**

- **Release/Close your resources (avoid memory leaks)**

- **Don't use functions that crash in case of bad input (gets, scanf, atoi, atol) use safe versions instead (fgets, sscanf, strtol).**

Entry Point

- **int main(int argc, char* argv[])**
 - defines the entry point to the application / process
 - return value represents an error code (EXIT_SUCCESS = 0, error > 0)
 - check return value in the shell using \$?
 - ./myprog
 - echo \$?
 - **argc is the argument counter, always >=1**
 - **argv[0] .. argv[argc-1] is a string array containing the parameters**
 - separated by space
 - argv[0] always refers to the application name
 - **Examples**
 - ./myprog test argc=2
 - ./myprog -o test argc=3
 - ./myprog -o foo bar argc=4
 - ./myprog -ofoo bar argc=3

Argument parsing

- **Further args can be used as options (first) or arguments**
 - E.g. `curl -X GET http://localhost:8080`
 - Here `-X` is a short option and can also be used as `--request` (long option) before the URL argument
 - Multiple options can be used together like
 - `ls -ld` which equals to `ls -l -d`
 - Options can appear only once
 - arguments to options are possible (optional space) like
 - `gcc -o myprog myprog.c`
 - `gcc -omyprog myprog.c`
- **Use getopt function for simpler parsing**
 - `getopt.h` needs to be included
 - function needs `argc`, `argv` and a format string with valid options

Argument parsing

- **int getopt (int argc, char *const argv[], const char *optstring)**
 - **optstring**
 - a letter without colon afterwards defines an option without argument
 - a letter with colon afterwards defines an option with mandatory argument
 - **EOF is returned as the last argument after looping through the options**
 - **global variable optarg points to argument of an option**
 - **global variable optind shows the index of next element to be processed in argv array**
 - **further examples:**
 - https://manpages.ubuntu.com/manpages/jammy/en/man3/getopt_long_only.3.html

Argument parsing example

```
1  int c;
2  int isAEnabled = 0;
3  while( (c = getopt(argc, argv, "af:")) != EOF ){
4      switch( c ){
5          case 'a': /* Option without argument */
6              isAEnabled++;
7              break;
8          case 'f': /* Option with argument */
9              printf("%s", optarg);
10             break;
11          case '?': /* illegal argument */
12              print_usage();
13          default: /* unmöglich */
14              assert( 0 );
15      }
16 }
```